CO3094 Phase 1 Backend Guide

This guide explains what the Phase 1 HTTP backend does, what you should learn before reading the code, and how the modified files work together.

1. What This Project Is

This project is a small HTTP backend server written with only the Python standard library.

It does not use Flask, FastAPI, Django, aiohttp, or any external framework. Instead, it builds the basic ideas of a web server manually:

- open a TCP socket
- accept a browser or curl connection
- read raw HTTP text
- parse the request
- dispatch it to either a static file or a route handler
- build a valid HTTP response
- send the response back through the socket

Phase 1 is synchronous. That means the server handles requests with normal blocking socket code and threads. Asyncio is intentionally not implemented yet.

2. Theory To Learn Before Reading The Code

2.1 TCP Sockets

HTTP runs on top of TCP.

A TCP server normally does this:

1. Create a socket.
2. Bind it to an IP and port.
3. Listen for clients.
4. Accept a connection.
5. Receive bytes.
6. Send bytes back.
7. Close the connection.

In this project, that logic lives mainly in:

- daemon/backend.py
- daemon/httpadapter.py

Important Python functions:

- socket.socket(...)
- bind(...)

- listen(...)
- accept(...)
- recv(...)
- sendall(...)
- close(...)

2.2 HTTP Request Format

When you run:

```
curl.exe -i http://127.0.0.1:9000/
```

curl sends a raw HTTP request similar to:

```
GET / HTTP/1.1
Host: 127.0.0.1:9000
User-Agent: curl/...
Accept: */*
```

An HTTP request has three main parts:

1. Request line
2. Headers
3. Optional body

Example with a body:

```
POST /hello HTTP/1.1
Host: 127.0.0.1:2026
Content-Type: application/json
Content-Length: 16

{"message":"hi"}
```

The first line tells the server:

- method: POST
- path: /hello
- HTTP version: HTTP/1.1

Headers give metadata. The body carries data for POST and PUT.

This project parses requests in:

  - daemon/request.py

2.3 HTTP Response Format

The server must send back a valid HTTP response:

```
HTTP/1.1 200 OK
Date: ...
Server: AsynapRous/1.0
Content-Length: 554
Connection: close
Content-Type: text/html; charset=utf-8

<html>...</html>
```

An HTTP response has:

1. Status line
2. Headers
3. Blank line
4. Optional body

Common status codes:

  - 200 OK: request succeeded
  - 400 Bad Request: request was malformed
  - 403 Forbidden: blocked path
  - 404 Not Found: file or route not found
  - 405 Method Not Allowed: HTTP method unsupported
  - 500 Internal Server Error: server code crashed

This project builds responses in:

  - daemon/response.py

2.4 HTTP Methods

The Phase 1 dispatcher supports:

  - GET: fetch a file or page
  - POST: send data to a route
  - PUT: update or submit data to a route
  - DELETE: request deletion, currently dispatch-supported but no sample route

HEAD is not implemented in Phase 1.

This logic is in:

- daemon/httpadapter.py

2.5 Static Files

Static files are normal files returned by the server.

Examples:

- / maps to www/index.html
- /css/styles.css maps to static/css/styles.css
- /images/welcome.png maps to static/images/welcome.png

The server must detect the MIME type:

- .html -> text/html
- .css -> text/css
- .png -> image/png
- .jpg -> image/jpeg

This logic is in:

- daemon/response.py

2.6 Routing

Routing means mapping a method and path to a Python function.

Example:

```
    @app.route('/hello', methods=['POST'])
    def hello(headers, body):
        ...
```

This creates a route like:

```
    ('POST', '/hello') -> hello
```

When a request arrives, the server checks:

```
    routes.get((request.method, request.path))
```

If a function exists, it calls the function.

This project has route registration in:

  - daemon/asynaprous.py

And route dispatch in:

  - daemon/httpadapter.py

2.7 JSON

JSON is a common format for API responses.

Example:

```
    {"message": "Welcome to the RESTful TCP WebApp"}
```

Python converts dictionaries to JSON with:

```
    json.dumps(data)
```

Sample route handlers return JSON bytes.

This project uses JSON in:

  - apps/sampleapp.py
  - daemon/response.py

2.8 Cookies And Query Parameters

Query parameters are in the URL:

```
    /hello?name=alice&age=20
```

Cookies are sent in headers:

```
    Cookie: sessionid=abc123; theme=dark
```

Phase 1 parses both so later phases can use them.

This logic is in:

- daemon/request.py

2.9 Logging

Logging helps you see what the server is doing.

Example logs:

```
[INFO] daemon.backend: Listening on 127.0.0.1:9000
[INFO] daemon.request: Parsed request method=GET path=/
[INFO] daemon.response: Built static response status=200
```

This is better than many random print(...) calls because logs can show module names and severity levels.

3. What I Changed

3.1 daemon/request.py

I completed request parsing.

It now parses:

- request line
- method
- path
- HTTP version
- headers
- body
- query parameters
- cookies
- route hook lookup

Important flow:

```
req.prepare(raw_http_message, routes)
```

Inside prepare(...), the request object fills these fields:

```
req.method
req.path
```

```
    req.version
    req.headers
    req.body
    req.query_params
    req.cookies
    req.hook
```

Example:

```
    POST /hello?name=alice HTTP/1.1
    Cookie: theme=dark

    {"message":"hi"}
```

becomes:

```
    method = "POST"
    path = "/hello"
    query_params = {"name": ["alice"]}
    cookies = {"theme": "dark"}
    body = '{"message":"hi"}'
```

3.2 daemon/response.py

I completed response building.

It now supports:

- proper HTTP status line
- response headers
- Content-Length
- Content-Type
- static files
- JSON responses
- text error responses
- MIME type detection
- path traversal protection

Important methods:

```
    build_response(...)
    build_json_response(...)
    build_error(...)
```

```
    build_notfound(...)
```

For static files:

```
    GET /
```

returns:

```
    www/index.html
```

For route output:

```
    POST /hello
```

returns JSON.

3.3 daemon/httpadapter.py

I completed synchronous request handling and route dispatch.

It now:

1. Reads bytes from the socket.
2. Reads the full body using Content-Length.
3. Parses the request.
4. Checks the HTTP method.
5. Serves static files for GET.
6. Calls route functions for POST, PUT, and DELETE.
7. Sends the response with sendall(...).
8. Closes the connection.

Supported methods:

```
    {"GET", "POST", "PUT", "DELETE"}
```

Async route functions are not used in Phase 1.

3.4 daemon/backend.py

I made the backend server launch correctly in synchronous mode.

It now:

  - binds to the requested IP and port
  - listens for connections
  - accepts clients
  - handles clients in threads
  - logs server activity
  - enables SO_REUSEADDR

start_backend.py uses this path:

```
    start_backend.py -> daemon.create_backend -> daemon.backend.run_backend
```

3.5 apps/sampleapp.py

I aligned the sample routes with your Phase 1 objective:

```
    POST /hello
    PUT /login
```

Expected sample outputs:

POST /hello:

```
    {"id": 1, "name": "Alice", "email": "alice@example.com"}
```

PUT /login:

```
    {"message": "Welcome to the RESTful TCP WebApp"}
```

3.6 daemon/dictionary.py

I fixed compatibility with newer Python versions.

Old code:

```
    from collections import MutableMapping
```

Newer Python expects:

```
from collections.abc import MutableMapping
```

The code now supports both old and new Python versions.

4. How The Files Work Together

For GET /:

```
curl -> socket -> backend.py -> httpadapter.py -> request.py
     -> response.py -> www/index.html -> curl
```

For POST /hello:

```
curl -> socket -> backend.py -> httpadapter.py -> request.py
     -> sampleapp.hello(...) -> response.py -> curl
```

For PUT /login:

```
curl -> socket -> backend.py -> httpadapter.py -> request.py
     -> sampleapp.login(...) -> response.py -> curl
```

5. How To Test

5.1 Test Static Backend

Terminal 1:

```
cd D:\CO3094-asynaprous
python start_backend.py --server-ip 127.0.0.1 --server-port 9000
```

Terminal 2:

```
curl.exe -i http://127.0.0.1:9000/
curl.exe -i http://127.0.0.1:9000/css/styles.css
curl.exe -i http://127.0.0.1:9000/images/welcome.png
```

5.2 Test Sample App Routes

Terminal 1:

```
cd D:\CO3094-asynaprous
python start_sampleapp.py --server-ip 127.0.0.1 --server-port 2026
```

Terminal 2:

```
curl.exe -i http://127.0.0.1:2026/
```

```
curl.exe -i -X POST http://127.0.0.1:2026/hello -H "Content-Type: application/json" -d
’{\"message\":\"hi\"}’
```

```
curl.exe -i -X PUT http://127.0.0.1:2026/login -H "Content-Type: application/json" -d
’{\"username\":\"alice\"}’
```

```
curl.exe -i -X DELETE http://127.0.0.1:2026/login
```

The DELETE request should return 404 Not Found unless you add a
DELETE /login route later. This still proves the dispatcher understands the
method.

6. Common Mistakes

Mistake 1: Testing The Wrong Port

start_backend.py default port:

```
9000
```

start_sampleapp.py default port:

```
2026
```

If you start the backend on 9000 but curl 2026, it will fail.

Mistake 2: Closing The Server Terminal

The server terminal must stay running.

If you see a new PowerShell prompt immediately after starting the server, the server exited or crashed.

Mistake 3: Broken PowerShell Quotes

If PowerShell shows:

```
    >>
```

your command has broken quotes. Press Ctrl+C and retype the curl command.

Mistake 4: Expecting POST On start_backend.py

start_backend.py serves static files only.

Use start_sampleapp.py for:

- POST /hello
- PUT /login

7. Reading Order For The Code

Read files in this order:

1. start_backend.py
2. daemon/backend.py
3. daemon/httpadapter.py
4. daemon/request.py
5. daemon/response.py
6. daemon/asynaprous.py
7. apps/sampleapp.py

This order follows the real execution path.

8. Mental Model

Think of the server as a pipeline:

```
    Raw bytes
```

```
      -> Request parser
      -> Router / static file lookup
      -> Handler function or file reader
      -> Response builder
      -> Raw bytes back to client
```

That is the core idea of almost every web framework. Flask and FastAPI hide these steps from you. This assignment asks you to build the steps yourself.

9. What Is Not Implemented Yet

Phase 1 intentionally does not implement:

- authentication
- sessions
- advanced cookie handling
- asyncio server mode
- async route execution
- database access
- middleware
- full REST application design
- HEAD support

Those can be later phases.

10. Summary

The Phase 1 backend is now a working synchronous HTTP server.

It can:

- accept TCP connections
- parse HTTP requests
- read headers and bodies
- parse query parameters and cookies
- serve static files
- return JSON
- dispatch GET, POST, PUT, and DELETE
- log server activity

The most important files are:

- daemon/backend.py: server loop
- daemon/httpadapter.py: connection handling and dispatch
- daemon/request.py: request parsing
- daemon/response.py: response formatting
- apps/sampleapp.py: example routes